//Program on Wrapper class object (Autoboxing)

```
package com.ravi.integral_literal;

public class Ex09
{
    public static void main(String[] args)
    {
        //Line 10 : static boxing
        Integer i = 12;
        Integer j = 24;
        Integer k = i + j;
        System.out.println("Sum is :"+k);

        Double d = 98.76;
        System.out.println(d);

        Character c = 'A';
        System.out.println(c);

        Boolean b = true;
        System.out.println(b);

    }
}
```

---

How to find the size, minimum value and maximum value of Integral Data type :

* The Wrapper classes like Byte, Short, Integer and Long have provided the following final and static variables to get the size, minimum value and maximum value of respective data types.

1) **SIZE** : It is a final and static variable available in the Wrapper classes to get the size of corresponding data type.

2) **MIN_VALUE** : It is a final and static variable available in the Wrapper classes to get the minimum value of the corresponding data type.

3) **MAX_VALUE** : It is a final and static variable available in the Wrapper classes to get the maximum value of the corresponding data type.

Example : If we want to find the size, minimum value and maximum value of byte data type then we should use the following code :

```
Byte.SIZE = 8 bits
Byte.MIN_VALUE = -128
Byte.MAX_VALUE = 127
```

```
package com.ravi.integral_literal;

public class Ex09
{
    public static void main(String[] args)
    {
        System.out.println("\t Byte Information :");
        System.out.println("Size :"+Byte.SIZE);
        System.out.println("Min Value :"+Byte.MIN_VALUE);
        System.out.println("Max Value :"+Byte.MAX_VALUE);

        System.out.println("-------------------------------");

        System.out.println("\t Short Information :");
        System.out.println("Size :"+Short.SIZE);
        System.out.println("Min Value :"+Short.MIN_VALUE);
        System.out.println("Max Value :"+Short.MAX_VALUE);

         System.out.println("-------------------------------");

        System.out.println("\t Integer Information :");
        System.out.println("Size :"+Integer.SIZE);
        System.out.println("Min Value :"+Integer.MIN_VALUE);
        System.out.println("Max Value :"+Integer.MAX_VALUE);

         System.out.println("-------------------------------");

        System.out.println("\t Long Information :");
        System.out.println("Size :"+Long.SIZE);
        System.out.println("Min Value :"+Long.MIN_VALUE);
        System.out.println("Max Value :"+Long.MAX_VALUE);
    }
}
```

---

floating point literal :

* If any numeric literal **contains decimal OR fraction** then it is called floating point literal.
  Example : 12.90, 0.5, 1.9

* In floating point literal we have 2 data types :
  a) float (32 bits)
  b) double (64 bits)

* Any floating point literal is of type double only so, the following statement will generate compilation error

```
float f1 = 0.1;//error
```

* We can assign floating point literal by using the following statements :

```
float f1 = (float).1;//Valid
float f2 = 0.2F;  //Valid
float f3 = 0.3f;  //Valid
```

* As we know by default, every floating point literal is of type **double** only but still java compiler has provided the following two flavors to represent the double value explicitly to enhance the readability of the code.

```
double d1 = 120;
double d2 = 900;
```

* Floating point literal we can represent in exponent form.

```
double d1 = 15e3;
double d2 = 15e-3;
```

* While working with Integral literal, We can represent it in different forms i.e Decimal, Octal, Hexa and binary but floating point literal we can represent in only one form i.e. decimal.

* We can assign integral literal (byte, short, int and long) to floating point literal but floating point literal (float and double) we cannot assign to integral literal.

//Programs :

```
public class Test
{
    public static void main(String[] args)
    {
        float f = 0.0;//error
        System.out.println(f);
    }
}

public class Test1
{
    public static void main(String[] args)
    {
        float b = 15.29F;
        float a = 15.29F;
        float d = (float) 15.30;

        System.out.println(b + " " +a+ " " +d);
    }
}

public class Test2
{
    public static void main(String[] args)
    {
        double d = 15.15;
        double e = 150;
        double f = 900;

        System.out.println(d+" ,"+e+" ,"+f);
    }
}

public class Test3
{
    public static void main(String[] args)
    {
        double x = 0129.89;

        double y = 0167;

        double z = 0178;//error

        System.out.println(x+" ,"+y+" ,"+z);
    }
}

class Test4
{
    public static void main(String[] args)
    {
        double x = 0X29;

        double y = 0X91.5;//error

        System.out.println(x+" ,"+y);
    }
}

public class Test5
{
    public static void main(String[] args)
    {
        double d1 = 15e3;
        System.out.println("d1 value is :"+d1);

        double d2 = 15e3;
        System.out.println("d2 value is :"+d2);
    }
}

public class Test6
{
    public static void main(String[] args)
    {
        double a = 0791;//error

        double b = 0791.0;

        double c = 0777;

        double d = 0X0ead;

        double e = 0X0ead.0;//error
    }
}

public class Test7
{
    public static void main(String[] args)
    {
        double a = 1.5e3;
        float b = 1.5e3;//error
        long c = 12.78;
        double d = 10;
        int e = 10.0;//error
        int f = 10.0F;//error
        int g = 10F;//error
        long l = 12.78F;//error
    }
}
```

//Range and size of floating point literal
public class Test8

```
{
    public static void main(String[] args)
    {
        System.out.println("\n Float range:");
        System.out.println("max :"+ Float.MAX_VALUE);
        System.out.println("max :"+ Float.MAX_VALUE);
        System.out.println(" size :"+Float.SIZE);

        System.out.println("\n Double range:");
        System.out.println(" min :" + Double.MIN_VALUE);
        System.out.println(" max :" + Double.MAX_VALUE);
        System.out.println(" size :"+Double.SIZE);
    }
}
```

---

* For static variable and static method, we need not to create any object. We can directly call with the help of class name if it is available in another class.

var keyword in java :

* It is introduced from JDK 10v
* We can use var keyword to create local variable with any kind of value.
  **Local variable type is decided by compiler at the time of declaration only.**
* var keyword we can use only for local variable.

```
package com.ravi.basic;
public class VarDemo
{
    public static void main(String[] args)
    {
        var i = 120;
        var j = 90;
        System.out.println(i);
    }
}
```

---

_ symbol in numbers literal :

* It is introduced from JDK 1.7V
* While writing the large numbers we can use _ symbol to separate the numbers hence the readability of the number will be increased.
* _ symbol can not be the first or last digit of the number.

```
package com.ravi.basic;

public class NumberVar
{
    public static void main(String[] args)
    {
        long mobile = 98_1234_5678L;
        System.out.println(mobile);
    }

}
```